



Chapter 4

Processing Text and Other Sequences

IN THIS CHAPTER

» Understanding natural language processing

» Considering raw text processing and use of sparse matrices

» Performing scoring and classification

.....

Natural Language Processing (NLP) is all about taking text that humans can understand as words, even if those words don't form a sentence, and putting it in a form that computers can process in some manner to look for patterns. For example, the computer doesn't understand "Turn on the radio," but it can process that command into a specific pattern. Sometimes, the computer also reacts to the processed pattern to perform a task, such as turning on the radio. The processing and the action of performing the task are separate. In this chapter, you start with the basics needed to understand NLP and see how it can serve you in building better applications for language problems. For example, you discover some of the issues in processing even raw text and in storing some types of data using sparse matrices so that the data doesn't take so much space. You also discover how to score and classify text.



REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0404_NLP.ipynb` source code file for Python and the `DSPD_R_0404_NLP.ipynb` source code file for R. See the Introduction for details on how to find these source files.

Introducing Natural Language Processing

When thinking about language in data science, you need to consider it from two perspectives: human and computer. The human perspective defines what people expect of each other in the way of communication, as well as what people perceive that the computer is doing, even if it isn't doing it at all. The computer perspective considers how the computer gives the appearance of working with language. The following sections introduce both perspectives of language.

Defining the human perspective as it relates to data science

Text comes in a number of forms. Most of these forms are language based, even the name of items such as the parts of a product. When you call a piece of metal bent in a particular way a widget and use that name to order it from a company, the term *widget* is part of a language devised to make communication possible. However, just saying *widget* wouldn't convey enough meaning; you must place the term in a sentence or provide context by placing the name on an order form. Even though humans understand that a part name isn't a sentence, a computer doesn't understand the difference at all. In fact, a computer can't understand language; it only processes language for specific applications.

As human beings, understanding language is one of our first achievements, and associating words to their meaning seems natural. It's also automatic to handle discourses that are ambiguous, unclear, or simply have a strong reference to the context of where we live or work (such as dialect, jargon, or terms that family or associates understand). In addition, humans can catch subtle references to feelings and sentiments in text, enabling people to understand polite speech that hides negative feelings and irony. Computers don't have these abilities but can rely on Natural Language Processing (NLP) — discussed in the next section.

Humans also resolve ambiguity by examining the text for hints about elements like place and time that express the details of the conversation (such as understanding what happened between John and Luca, or whether the conversation is about a computer when mentioning a mouse. Relying on additional information for understanding is part of the human experience. This sort of analysis is somewhat difficult for computers. Moreover, if the task requires critical contextual knowledge or demands

that the listener resort to common sense and general expertise, the task for computers becomes daunting.

Considering the computer perspective as it relates to data science

Textual language elements include text-based numeric information and special symbols. It can also include certain kinds of encodings. A computer can perform math only on actual numbers (rather than on a category or text), and math is the only basis on which computers can process anything. The string 1 is different from the number 1 as far as the computer is concerned. A string 1 appears as the number 49 to the computer when using the ASCII encoding, while the numeric 1 is precisely that — a 1. Any text provided as input, regardless of whether a human considers it to be text, requires processing before the computer can perform math calculations (including comparisons) on it.



REMEMBER A computer can't normally process language at all unless it's highly formal and precise, such as a programming language. Rigid syntax rules and grammar enable a computer to turn a program written by a developer in a computer language like Python into the machine language using a compiler or interpreter (special kinds of programs) that determines what tasks the computer will perform. The fact that most humans can barely read a program doesn't change the fact that a program is a kind of human language. The machine language that a computer processes is numbers consisting of opcodes (what to do) and operands (the data required to perform the task). You can see a listing of such numbers for Intel processors at

<http://www.mathemainzel.info/files/x86asmref.html>. A human sees nothing but numbers; the computer sees a set of precise instructions telling it what to do and how to do it.

However, this chapter isn't about programming languages that are relatively easy to process because of the rigidity of their nature. Instead, it deals with human language of the sort used in conversation, both spoken and written. Human language is not at all similar to a computer's language because it

- Lacks a precise structure
- Is full of errors
- Contains contradictions
- Hides meaning in ambiguities

Yet human language works well for humans, with some effort on the part of the hearer, to serve human society and the progress of knowledge. Humans can overcome the deficiencies of the spoken or written word with relative ease; it's something that we do based on years of experience and a few errors in childhood. This kind of processing is outside the reach of a computer, however, even if it were to have a special compiler to help handle the task.

Since Alan Turing first devised the Turing Test in 1950, which aims at spotting an artificial intelligence based on how it communicates with humans (https://en.wikipedia.org/wiki/Turing_test), NLP experts have developed a series of techniques that define the state of the art in computer-human interaction by text. Simply put, NLP still has a lot of ground to cover in order to discover how to extract meaningful summaries from text effectively or how to complete missing information from text. Programming a computer to process human language is therefore a daunting task, which is only recently possible using the following:

- Natural Language Processing (NLP)
- Deep learning Recurrent Neural Networks (RNNs)
- Word embeddings



TECHNICAL
STUFF

Word embeddings is the name of the language-modeling and feature-learning technique in NLP that maps vocabulary to real number vectors using products like Word2vec, GloVe, and FastText. You also see it used in most recent state-of-the-art networks such as Google's open-sourced BERT that uses embeddings for words based on the context they appear in, allowing better performing NLP solutions.

A computer powered with NLP can successfully spot spam in your email, tag the part of a conversation that contains a verb or a noun, and spot an entity like the name of a person or a company (called *named entity recognition*; see <https://medium.com/explore-artificial-intelligence/introduction-to-named-entity-recognition->

`eda8c97c2db1`). All these achievements have found application in tasks such as spam filtering, predicting the stock market using news articles, and de-duplicating redundant information in data storage.

Things get more difficult for NLP when translating a text from another language and understanding whom or what the subject is in an ambiguous phrase. For example, consider the sentence, “John told Luca he shouldn’t do that again.” In this case, you can’t really tell whether “he” refers John or Luca. Disambiguating words with many meanings, such as considering whether the word *mouse* in a phrase refers to an animal or a computer device, can prove difficult. Obviously, the difficulty in all these problems arises because of the context.

Understanding How Machines Read

Before a computer can do anything with text, it must be able to read the text in some manner. Book 3, [Chapter 1](#) shows how you can prepare data to deal with categorical variables, such as a feature representing a color (for instance, representing whether an example relates to the colors red, green, or blue). Categorical data is a type of short text that you represent using binary variables, that is, variables coded using one or zero values according to whether a certain value is present in the categorical variable. Not surprisingly, you can represent complex text using the same logic.

Creating a corpus

Just as you transform a categorical color variable, having values such as red, green, and blue, into three binary variables, with each one representing one of the three colors, so you can transform a phrase like “The quick brown fox jumps over the lazy dog” using nine binary variables, one for each word that appears in the text.

When working with categories, “The” is distinct from “the” because of its initial capital letter. This is the bag-of-words (BoW) form of representation. In its simplest form, BoW shows whether a certain word is present in the text by flagging a specific feature in the dataset.

The input data is three phrases, `text_1`, `text_2`, and `text_3`, placed in a list, `corpus`. A *corpus* is a set of homogenous documents, put together

for NLP analysis:

```
text_1 = 'The quick brown fox jumps over the lazy dog.'
```

```
text_2 = 'My dog is quick and can jump over fences.'
```

```
text_3 = 'Your dog is so lazy that it sleeps all the day.'
```

```
corpus = [text_1, text_2, text_3]
```



TIP

When you analyze text using a computer, you load the documents from disk or scrape them from the web and place each of them into a string variable. If you have multiple documents, you store them all in a list, called the corpus. When you have a single document, you can split it using chapters, paragraphs, or simply the end of each line. After splitting the document, place all its parts into a list and apply analysis as if the list were a corpus of documents.

Performing feature extraction

Now that you have a corpus, you use a class from the `feature_extraction` module in Scikit-learn, `CountVectorizer`, which easily transforms texts into BoW, like this:

```
from sklearn.feature_extraction import text
```

```
vectorizer = text.CountVectorizer(binary=True).fit(corpus)
```

```
vectorized_text = vectorizer.transform(corpus)
```

```
print(vectorized_text.todense())
```

The output is a list of lists, as shown here:

```
[[0 0 1 0 0 1 0 1 0 0 0 1 1 0 1 1 0 0 0 1 0]]
```

```
[0 1 0 1 0 1 1 0 1 0 1 0 0 1 1 1 0 0 0 0 0]]
```

```
[1 0 0 0 1 1 0 0 1 1 0 0 1 0 0 0 1 1 1 1 1]]
```

The `CountVectorizer` class learns the corpus content using the `fit` method and then turns it (using the `transform` method) into a list of lists. A list of lists is nothing more than a matrix in disguise, so what the class returns is actually a matrix made of three rows (the three documents, in the same order as the corpus) and 21 columns representing the content.



WARNING When viewing the output of this part of the code, remember that the output is zero based. Consequently, the first entry in the list has a value of 0, not 1.

Understanding the BoW

The BoW representation turns words into the column features of a document matrix, and these features have a nonzero value when present in the processed text. For instance, consider the word *dog*. The following code shows its representation in the BoW:

```
print(vectorizer.vocabulary_)
```

The straightforward output appears here:

```
{'day': 4, 'jumps': 11, 'that': 18, 'the': 19, 'is': 8,
```

```
'fences': 6, 'lazy': 12, 'and': 1, 'quick': 15, 'my': 13,
```

```
'can': 3, 'it': 9, 'so': 17, 'all': 0, 'brown': 2,
```

```
'dog': 5, 'jump': 10, 'over': 14, 'sleeps': 16,
```

```
'your': 20, 'fox': 7}
```

Of course, if you have a larger vocabulary, you want it sorted. The following code will show a sorted version for you:

```
from collections import OrderedDict
```

```
ordered = OrderedDict(sorted(
```

```
vectorizer.vocabulary_.items(),
```

```
key=lambda x: x[1]))
```

```
print(dict(ordered))
```

The output now appears as

```
{'all': 0, 'and': 1, 'brown': 2, 'can': 3, 'day': 4,
```

```
'dog': 5, 'fences': 6, 'fox': 7, 'is': 8, 'it': 9,
```

```
'jump': 10, 'jumps': 11, 'lazy': 12, 'my': 13,
```

```
'over': 14, 'quick': 15, 'sleeps': 16, 'so': 17,
```

```
'that': 18, 'the': 19, 'your': 20}
```

Asking the `CountVectorizer` to print the vocabulary it learned from the sample text outputs a report that it associates *dog* with the number 5, which means that *dog* is the sixth element in the BoW representations. In fact, in the obtained BoW, the sixth element of each document list always has a value of 1 because *dog* is the only word present in all three documents.



REMEMBER Storing documents in a document matrix form can be memory intensive because you must represent each document as a vector of the same length as the dictionary that created it. The dictionary in this example is quite limited, but when you use a larger corpus, you discover that a dictionary of the English language contains well over a million terms. The solution is to use sparse matrices. A *sparse matrix* is a way to store a matrix in your computer's memory without having zero values occupying memory space. You can read more about sparse matrices here:

https://en.wikipedia.org/wiki/Sparse_matrix.

Processing and enhancing text

Marking whether a word is present in a text is a good start on machine reading, but sometimes it's not enough. The BoW model has its own limits. In a BoW, words lose their order relationship with each other; it's like putting stuff randomly into a bag. For instance, in the phrase *My dog is*

quick and can jump over fences, you know that *quick* refers to *dog* because it is glued to it by *is*, a form of the verb *to be*. In a BoW, however, everything is mixed and some internal references are lost. Further processing can help prevent the reference losses. The following sections discuss how to process and enhance text.

Performing word counting

Instead of marking the presence or absence of an element of the phrase (technically called a *token*), you can instead count how many times it occurs, as shown in the following code:

```
text_4 = 'A black dog just passed by but my dog is brown.'
```

```
corpus.append(text_4)
```

```
vectorizer = text.CountVectorizer().fit(corpus)
```

```
vectorized_text = vectorizer.transform(corpus)
```

```
print(vectorized_text.todense()[-1])
```

When you run this code, you get the following output:

```
[[0 0 1 1 1 1 0 0 2 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 0]]
```

This code modifies the previous example by adding a new phrase with the word *dog* repeated two times. The code appends the new phrase to the `corpus` and retrains the `vectorizer`, but it omits the `binary=True` setting this time. The resulting vector for the last inserted document clearly shows a `2` value in the ninth position, thus the `vectorizer` counts the word *dog* twice.

Counting tokens helps make important words stand out. Yet, it's easy to repeat phrase elements, such as articles, that aren't important to the meaning of the expression. In the “[Stemming and removing stop words](#)” section, you discover how to exclude less important elements, but for the time being, the example underweights them using the Term Frequency-Inverse Document Frequency (TF-IDF) transformation.

Changing weights using TF-IDF

The TF-IDF transformation is a technique that, after counting how many times a token appears in a phrase, divides the value by the number of documents in which the token appears. Using this technique, the vectorizer deems a word less important, even if it appears many times in a text, when it also finds that word in other texts. In the example corpus, the word *dog* appears in every text. In a classification problem, you can't use the word to distinguish between texts because it appears everywhere in the corpus. The word *fox* appears in only one phrase, making it an important classification term.

You commonly apply a number of transformations when applying TF-IDF, with the most important transformation normalizing the text length. Clearly, a longer text has more chances to have more words that are distinctive when compared to a shorter text. For example, when the word *fox* appears in a short text, it appears relevant to the meaning of that expression because *fox* stands out among few other words. However, having *fox* appear once in a long text might not seem to matter as much because it's a single word among many others. For this reason, the transformation divides the total tokens by the count of each token for each phrase. Treating a phrase like this turns token counting into a token percentage, so TF-IDF no longer considers how many times the word *fox* appears, but instead considers the percentage of times the word *fox* appears among all the tokens. The following example demonstrates how to complete the previous example using a combination of normalization and TF-IDF:

```
Tfidf = text.TfidfTransformer(norm='l1')
```

```
tfidf = Tfidf.fit_transform(vectorized_text)
```

```
phrase = 3 # choose a number from 0 to 3
```

```
total = 0
```

```
for word in vectorizer.vocabulary_:
```

```
    pos = vectorizer.vocabulary_[word]
```

```
    value = list(tfidf.toarray()[phrase])[pos]
```

```
    if value != 0:
```

```
print ("%10s: %0.3f" % (word, value))
```

```
total += value
```

```
print ('\nSummed values of a phrase: %0.1f' % total)
```

When you run this code, you see the following output:

```
is: 0.077
```

```
by: 0.121
```

```
brown: 0.095
```

```
dog: 0.126
```

```
just: 0.121
```

```
my: 0.095
```

```
black: 0.121
```

```
passed: 0.121
```

```
but: 0.121
```

```
Summed values of a phrase: 1.0
```

Using this new TF-IDF model rescales the values of important words and makes the number of entries comparable between each text in the corpus.

Maintaining order using n-grams

Your text started out in a particular order and with various aids to make it readable. The current text ordering isn't useful to humans because the words appear in an order that makes analysis possible. To recover part of the ordering of the text before the BoW transformation, you add n-grams (<https://en.wikipedia.org/wiki/N-gram>). An *n-gram* is a continuous sequence of tokens in the text that you use as a single token in the BoW

representation. For instance, in the phrase *The quick brown fox jumps over the lazy dog*, a *bi-gram* (pronounced “by gram”) — that is, a sequence of two tokens — transforms *brown fox* and *lazy dog* into single tokens. A *tri-gram* may create a single token from *quick brown fox*.

An n-gram is a powerful tool, but it has a drawback because it doesn’t know which combinations are important to the meaning of a phrase. N-grams create all the contiguous sequences of size N . The TF-IDF model can underweight the less useful n-grams, but only projects like Google’s NGram viewer (you can read more about this viewer later in the chapter) can tell you which n-grams are useful in NLP with any certainty. The following example uses `CountVectorizer` to model n-grams in the range of (2, 2), that is, bi-grams:

```
bigrams = text.CountVectorizer(ngram_range=(2,2))
```

```
ord_bigrams = OrderedDict(sorted(
```

```
    bigrams.fit(corpus).vocabulary_.items(),
```

```
    key=lambda x: x[1]))
```

```
print (dict(ord_bigrams))
```

Here is the output you see when running the example (in sorted order):

```
{'all the': 0, 'and can': 1, 'black dog': 2,
```

```
'brown fox': 3, 'but my': 4, 'by but': 5,
```

```
'can jump': 6, 'dog is': 7, 'dog just': 8,
```

```
'fox jumps': 9, 'is brown': 10, 'is quick': 11,
```

```
'is so': 12, 'it sleeps': 13, 'jump over': 14,
```

```
'jumps over': 15, 'just passed': 16, 'lazy dog': 17,
```

```
'lazy that': 18, 'my dog': 19, 'over fences': 20,
```

```
'over the': 21, 'passed by': 22, 'quick and': 23,
```

```
'quick brown': 24, 'sleeps all': 25, 'so lazy': 26,
```

```
'that it': 27, 'the day': 28, 'the lazy': 29,
```

```
'the quick': 30, 'your dog': 31}
```

If you don't care about order, you could simply print out the list of bi-grams, like this:

```
print (bigrams.fit(corpus).vocabulary_)
```



TIP

Setting different ranges lets you use both *unigrams* (single tokens) and n-grams in your NLP analysis. For instance, the setting `ngram_range=(1,3)` creates all tokens, all bi-grams, and all tri-grams. You usually never need more than tri-grams in an NLP analysis. Increasing the number of n-grams is slightly beneficial after trigrams and sometimes even just after bi-grams, depending on the corpus size and the NLP problem.

Stemming and removing stop words

Stemming is the process of reducing words to their stem (or root) word. This task isn't the same as understanding that some words come from Latin or other roots, but instead makes similar words equal to each other for the purpose of comparison or sharing. For example, the words *cats*, *catty*, and *catlike* all have the stem *cat*. The act of stemming helps you analyze sentences when tokenizing them because words having the same stem should have the same meaning (represented by a single feature).

GETTING THE NATURAL LANGUAGE TOOLKIT (NLTK)

This example requires the use of the Natural Language Toolkit (NLTK), which Anaconda doesn't install by default. The easy way to get this toolkit is to open the Anaconda prompt, ensure that you're in the (base) environment, and type this command:

```
conda install -c anaconda nltk
```

The conda command may appear to freeze because collecting the required information can take several minutes. Eventually, you see a message saying that conda is collecting the package metadata and you know that the installation is going as planned. Just be patient.

If you're working in an environment that doesn't match the book's environment, you must download and install NLTK using the instructions found at <http://www.nltk.org/install.html> for your platform. Make certain that you install the NLTK for whatever version of Python you're using for this book when you have multiple versions of Python installed on your system. After you install NLTK, you must also install the packages associated with it. The instructions at <http://www.nltk.org/data.html> tell you how to perform this task. (Install all the packages to ensure that you have everything.)

Creating stem words by removing suffixes to make tokenizing sentences easier isn't the only way to make the document matrix simpler.

Languages include many glue words that don't mean much to a computer but have significant meaning to humans, such as *a*, *as*, *the*, *that*, and so on in English. They make the text flow and concatenate in a meaningful way. Yet, the BoW approach doesn't care much about how you arrange words in a text. Thus, removing such words is legitimate. These short, less useful words are called *stop words*.



REMEMBER The act of stemming and removing stop words simplifies the text and reduces the number of textual elements so that only the essential elements remain. In addition, you keep just the terms that are nearest to the true sense of the phrase. By reducing the number of tokens, a computational algorithm can work faster and process the text more effectively when the corpus is large.

The following example demonstrates how to perform stemming and remove stop words from a sentence. It begins by training an algorithm to perform the required analysis using a test sentence. Afterward, the example checks a second sentence for words that appear in the first:

```
from sklearn.feature_extraction import text
```

```
import nltk

from nltk import word_tokenize

from nltk.stem.porter import PorterStemmer

nltk.download('punkt')

stemmer = PorterStemmer()

def stem_tokens(tokens, stemmer):

    stemmed = []

    for item in tokens:

        stemmed.append(stemmer.stem(item))

    return stemmed

def tokenize(text):

    tokens = word_tokenize(text)

    stems = stem_tokens(tokens, stemmer)

    return stems

vocab = ['Sam loves swimming so he swims all the time']

vect = text.CountVectorizer(tokenizer=tokenize,

                             stop_words='english')

vec = vect.fit(vocab)
```

```
sentence1 = vec.transform(['George loves swimming too!'])
```

```
print (vec.get_feature_names())
```

```
print (sentence1.toarray())
```

At the outset, the example creates a vocabulary using a test sentence and places it in the variable `vocab`. It then creates a `CountVectorizer`, `vect`, to hold a list of stemmed words, but it excludes the stop words. The `stop_words` parameter refers to a pickle file that contains the English stop words. The `tokenizer` parameter defines the function used to stem the words. The vocabulary is fitted into another `CountVectorizer`, `vec`, which is used to perform the actual transformation on a test sentence using the `transform()` function. (You can see other parameters for `CountVectorizer()` at

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

Here's the output from this example (note that the data path information may differ on your system).

```
[nltk_data] Downloading package punkt to
```

```
[nltk_data] C:\Users\Luca\AppData\Roaming\nltk_data...
```

```
[nltk_data] Unzipping tokenizers\punkt.zip.
```

```
['love', 'sam', 'swim', 'time']
```

```
[[1 0 1 0]]
```

WORKING WITH PYTHON PICKLES

No, Python developers aren't any more prone to eating condiments than anyone else is. A pickle is special kind of serialized storage format that you can read about at <https://pythontips.com/2013/08/02/what-is-pickle-in-python/>. Essentially, the pickle is a kind of package. It con-

tains stop words for a specific language, which is English in this case. It also has files for other languages, such as French and German (see the list at <https://pypi.org/project/stop-words/>). The pickle files can come with issues for English, as discussed at https://scikit-learn.org/stable/modules/feature_extraction.html#stop-words and <https://aclweb.org/anthology/W18-2502> .

The first output shows the stemmed words. Notice that the list contains only *swim*, not *swimming* or *swims*. All the stop words are missing as well. For example, you don't see the words *so*, *he*, *all*, or *the*.

The second output shows how many times each stemmed word appears in the test sentence. In this case, a *love* variant appears once and a *swim* variant appears once as well. The words *sam* and *time* don't appear in the second sentence, so those values are set to 0.

Scraping textual datasets from the web

Given NLP's capabilities, building complete language models is just a matter of gathering large text collections. Digging through large amounts of text enables machine learning algorithms using NLP to discover connections between words and derive useful concepts relative to specific contexts. For instance, when discussing a mouse in the form of a device or an animal, a machine learning algorithm powered by NLP text processing can derive the precise topic from other hints in the phrase. Humans decipher these hints by having lived, seen, talked about, or read about the topic of the conversation. The following sections discuss how computers interact with the web to develop better NLP capabilities.

Understanding web scraping

Computers also build information databases based on digital textual sources or inputs from sensors. As noted previously, computers don't see or hear as humans do, but only accept input in the form of data. The web offers access to millions of documents, most of them freely accessible without restrictions.



REMEMBER *Web scraping* (the act of accessing text on websites and downloading it as a data source) allows machine learning algorithms to automatically feed NLP processes and build new capabilities in recognizing and classifying text. Developers have already done much to create NLP systems capable of processing textual information better by leveraging the richness of the web. The developer creatively directs the computer to process data in specific ways, and the computer then does the following:

- Automates processing
- Ensures that processing is done consistently
- Presents new patterns that the developer must recognize as the result of processing

By using free text acquired from the web and other open text sources, such as dictionaries, scientists at Microsoft Research have developed various versions of MindNet, a semantic network that is a network of words connected by meaning. MindNet can find related words through the following:

- Synonyms
- Parts
- Causes
- Locations
- Sources

For instance, when you ask for the word *car*, MindNet provides answers such as *vehicle* (a synonym) and then connects *vehicle* to *wheel* because it is a specific part of a car, thus providing knowledge directly derived from text even though nobody has specifically instructed MindNet about cars or how they're made. You can read more about MindNet at

<https://research.microsoft.com/en-us/projects/mindnet/default.aspx>.

Google developed something similar based on its Google Books project, helping to build better language models for all Google's applications. A public API based on Google's work is the Ngram Viewer, which can explore how frequently certain combinations of tokens up to five-grams have appeared over time: <https://books.google.com/ngrams>.

The capability to retrieve information from the web allows even greater achievements. For example, you could build a dictionary of positive or negative words based on associated emoticons or emoji (<https://en.wikipedia.org/wiki/Emoji>).

Web scraping is a complex subject that could require an entire book to explain. This chapter offers an example of web scraping and an overview of what to expect.

Installing the BeautifulSoup package

You need to install the BeautifulSoup package when using Python to perform web scraping

(<http://www.crummy.com/software/BeautifulSoup/>). This package should already be part of your Anaconda installation. To determine whether you have BeautifulSoup installed, enter the following command at the Anaconda Prompt:

```
conda search beautifulsoup4 --info
```

After you type this command at the Anaconda Prompt and press Enter, you see output like this when the package is installed:

```
beautifulsoup4 4.7.1 py37_1001
```

```
-----
```

```
file name      : beautifulsoup4-4.7.1-py37_1001.tar.bz2
```

```
name           : beautifulsoup4
```

```
version        : 4.7.1
```

```
build          : py37_1001
```

```
build number   : 1001
```

```
size           : 140 KB
```

```
license        : MIT
```

```
subdir         : win-64
```

```
url      : https://conda.anaconda.org/conda-forge/...
```

```
-py37_1001.tar.bz2
```

```
md5      : 38cb1888a82d4a041fb5f7c087c66ff4
```

```
timestamp : 2019-01-08 22:44:20 UTC
```

```
dependencies:
```

```
- python >=3.7,<3.8.0a0
```

```
- soupsieve
```

The precise presentation can vary by system, but this gives you a good idea of what to look for. When Beautiful Soup is missing, you can easily install it on your system by typing this command at the Anaconda Prompt and pressing Enter:

```
conda install -c anaconda beautifulsoup4
```

Beautiful Soup, created by Leonard Richardson, is an excellent tool for scraping data from HTML or XML files retrieved from the web, even if they are malformed or written in a nonstandard way. The package name refers to the fact that HTML documents are made of tags, and when they are a mess, many developers idiomatically call the document a *tag soup*. Thanks to Beautiful Soup, you can easily navigate in a page to locate the objects that matter and extract them as text, tables, or links.

Using Beautiful Soup in your code

This example demonstrates how to download a table from a Wikipedia page containing all the major U.S. cities. Wikipedia (<https://www.wikipedia.org/>) is a free-access and free-content Internet encyclopedia, enjoyed by millions of users every day, all around the world. Because its knowledge is free, open, and, most important, well structured, it's a precious resource for learning from the web.

```
from bs4 import BeautifulSoup
```

```
import pandas as pd
```

```
try:

import urllib2 # Python 2.7.x

except:

import urllib.request as urllib2 # Python 3.x

wiki = "https://en.wikipedia.org/wiki/\
List_of_United_States_cities_by_population"

header = {'User-Agent': 'Mozilla/5.0'}

query = urllib2.Request(wiki, headers=header)

page = urllib2.urlopen(query)

soup = BeautifulSoup(page, "lxml")
```

After you upload the BeautifulSoup package, the code defines a header (stating that you are a human user using a browser) and a target page.

The target page is a document containing a list of major U.S. cities:

https://en.wikipedia.org/wiki/List_of_United_States_cities_by_population.

The list also contains information about the population and surface of the city.

```
table = soup.find("table",

{ "class" : "wikitable sortable" })

final_table = list()

for row in table.findAll('tr'):

cells = row.findAll("td")

if len(cells) >=6:

v1 = cells[1].find(text=True)
```

```
v2 = cells[2].find(text=True)
```

```
v3 = cells[3].find(text=True)
```

```
v4 = cells[4].find(text=True)
```

```
v5 = cells[6].findAll(text=True)
```

```
final_table.append([v1, v2, v3, v4, v5])
```

```
cols = ['City', 'State', 'Population_2017', 'Census_2010'
```

```
, 'Land_Area_km2']
```

```
df = pd.DataFrame(final_table, columns=cols)
```

```
print(df[['City', 'Population_2017']])
```

WIKIPEDIA CAVEATS

Most publishers and many college instructors view Wikipedia as being a dubious source of information. Anyone can edit the entries it contains, and sometimes people do so in ways that slant the information politically or socially, or simply reflects a lack of knowledge (see

<https://www.foxbusiness.com/features/just-how-accurate-is-wikipedia>). These issues can mean that the information you receive may

not reflect reality. However, many studies show that the community effort behind creating Wikipedia (see

<https://www.livescience.com/32950-how-accurate-is-wikipedia.html> , <https://www.cnet.com/news/study-wikipedia-as-accurate-as-britannica/> , and

<https://www.zmescience.com/science/study-wikipedia-25092014/>) does tend to mitigate this issue partially.

No matter which side of the Wikipedia as an information source divide you fall on, you need to exercise some level of care in taking Wikipedia entries at face value, just as you would any Internet content. Just because someone tells you that something is so doesn't make it true (no matter what form that information source might take). You need to cross-refer-

ence the information and verify the facts before accepting any Internet information source as factual, even Wikipedia. This said, the authors have verified every Wikipedia source used in the book as much as possible to ensure that you receive accurate information.

Wikipedia also has its own rules and terms of service, which you may read at

https://meta.wikimedia.org/wiki/Bot_policy#Unacceptable_usage.

The terms of service forbid the use of bots for automated tasks, such as modifying the website (corrections and automatic posting), and bulk downloads (downloading massive amounts of data). Wikipedia is a great source for NLP analysis because you can download all its English articles at <https://dumps.wikimedia.org/enwiki/>. Other languages are also available for download. Just consult <https://dumps.wikimedia.org/> for further information.

After downloading the page into the variable named `soup`, you can use the `find()` and `findAll()` methods to look for a table (the `<tr>` and `<td>` tags). The `cells` variable contains a number of cell entries, each of which can contain text. The code looks inside each cell for textual information (`v1` through `v5`) that it stores in a list (`final_table`). It then turns the list into a pandas `DataFrame` for further processing later. For example, you can use the `DataFrame`, `df`, to turn strings into numbers. Simply printing `df` outputs the resulting table. Here is an example of what you see from the `print()` call, which selects only two of the available columns:

	City	Population_2017
0	New York City	8,622,698
1	Los Angeles	3,999,759
2	Chicago	2,716,450
3	Houston	2,312,717
4	Phoenix	1,626,078
5	Philadelphia	1,580,863

...		
308	Tuscaloosa	100,287
309	San Angelo	100,119
310	Vacaville	100,032
[311 rows x 2 columns]		

Handling problems with raw text

Even though raw text wouldn't seem to present a problem in parsing because it doesn't contain any special formatting, you do have to consider how the text is stored and whether it contains special words within it. Some character sets don't quite match up, and converting between them can be difficult even when you don't think a problem should occur. Languages also use special character sets, and it can be hard to get anything but garbage when you try to transition the text to a form you can use within Python. The multiple forms of encoding on web pages can present interpretation problems that you need to consider as you work through the text. The following sections take a very brief look at some of the issues you'll encounter when working with supposedly raw text.

Dealing with encoding

The way the text is encoded can differ because of different operating systems, languages, and geographical areas. Be prepared to find a host of different encodings as you recover data from the web. Human language is complex, and the original ASCII coding, comprising just unaccented English letters, can't represent all the different alphabets. That's why so many encodings appeared with special characters. For example, a character can use either seven or eight bits for encoding purposes. The use of special characters can differ as well. In short, the interpretation of bits used to create characters differs from encoding to encoding. You can see a host of encodings at

<http://www.i18nguy.com/unicode/codepages.html>. Fortunately, Python supports a huge number of encodings, as shown at

<https://docs.python.org/3/library/codecs.html#standard-encodings>.



REMEMBER Sometimes you need to work with encodings other than the default encoding set within the Python environment. When working with Python 3.x, you rely on Universal Transformation Format 8-bit (UTF-8) as the encoding used to read and write files by default. To check the default encoding, you use this code:

```
import sys
```

```
sys.getdefaultencoding()
```

In previous versions of Python, you had to go through all sorts of weird machinations to read other file formats. The discussions at <https://stackoverflow.com/questions/2276200/changing-default-encoding-of-python> and <https://anonbadger.wordpress.com/2015/06/16/why-sys-setdefault-encoding-will-break-code/> will give you some idea of how bad things got. The point is that these old systems are no longer in place because you have easier options to use now. To encode and decode between the default Python UTF-8 and other encodings, you use the `encode()` and `decode()` methods, as shown here:

```
utf8_string = "Hello there!"
```

```
utf7_string = utf8_string.encode('utf7')
```

```
print(utf7_string, type(utf7_string))
```

```
utf7_string = "This is a new string!".encode("utf7")
```

```
utf8_string = utf7_string.decode('utf8')
```

```
print(utf8_string, type(utf8_string))
```

The output from this example code tells you something about Python:

```
b'Hello there!' <class 'bytes'>
```

```
This is a new string! <class 'str'>
```



REMEMBER When a string uses an encoding other than UTF-8, Python stores it as a series of bytes. You can still see the string as output, but you may not be able to read every character if Python can't read it correctly. You must also encode and decode the strings as necessary to ensure that the strings remain readable. For example, if you encode the first string as UTF-32 instead, what you see for output is

```
b'\xff\xfe\x00\x00H\x00\x00\x00e\x00\x00\x001\x00\x00\x001
```

```
\x00\x00\x00o\x00\x00\x00\x00\x00\x00t\x00\x00\x00h\x00
```

```
\x00\x00e\x00\x00\x00r\x00\x00\x00e\x00\x00\x00!\x00\x00
```

```
\x00'
```

If you look carefully, you can still see the individual characters, such as H as `\x00H` (along with some padding). The biggest problem with Python and encoding is that Python will encode and decode text using whatever encoding you specify, which means that you might end up with garbage. The data is there; you just can't read it. One way to overcome this issue is to try to determine the encoding based on locale, like this:

```
import locale
```

```
locale.getpreferredencoding()
```

Unfortunately, knowing the locale doesn't always help or solve problems. For example, when you run this code on a Windows system, you discover that Windows relies on `cp1252`, where `cp` stands for code page. Oddly enough, problems occur when transitioning `cp1252` to UTF-8, as discussed at <https://stackoverflow.com/questions/26324622/what-characters-do-not-directly-map-from-cp1252-to-utf-8>. The most important issue, though, is for you to know what sort of encoding the text uses before you try to process it in Python.

Considering Unicode

Languages include all sorts of accented characters, and the Unicode character set also has a wealth of special characters. To work with certain characters, you must rely on Unicode encoding, as described at <https://docs.python.org/3/howto/unicode.html>. The problem is that you must know which kind of encoding to use. Consider the following code:

```
uString1 = "This is a winking face: \N{WINKING FACE}"
```

```
print(uString1)
```

```
uString2 = "This is a winking face: \U0001F609"
```

```
print(uString2)
```

```
uString3 = "This is not a winking face: \u1F609"
```

```
print(uString3)
```



REMEMBER This is the three kinds of encoding that Python supports. The first two encodings, `name` and `32-bit`, produce the desired output of a winking face. Of course, you have to know that the character is named `WINKING FACE` to use the first encoding, and you have to know the actual number of the character to use the second encoding, but the encoding is possible and works without problem. The 16-bit encoding won't work in this case because the character number is too high. So, you must choose the correct encoding to get the desired result. Python outputs a spade symbol and the number 9 in the third case.

A problem can occur when you try to encode and decode strings with Unicode characters. For example, the following code produces less than stellar results:

```
utf7_string = uString1.encode('utf7')
```

```
print(utf7_string)
```

```
uString4 = utf7_string.decode('utf8')
```

```
print(uString4)
```

You might think that the string will become impossible to read when you encode it as UTF-7 but that decoding it should fix the problem. Unfortunately, the results are something different:

```
b'This is a winking face: +2D3eCQ-'
```

```
This is a winking face: +2D3eCQ-
```

The encoding and decoding processes produce unusable results. The problem can become worse when you try to encode `uString1` or `uString2` as ASCII. In this case, Python actually raises an example. However, you have a perfectly usable alternative in this case, as shown in the following code:

```
utf7_string = uString1.encode('ascii', 'namereplace')
```

```
print(utf7_string)
```

```
uString4 = utf7_string.decode('utf8', 'replace')
```

```
print(uString4)
```

The output still doesn't quite recover the original content, but at least you know what the original content was:

```
b'This is a winking face: \\N{WINKING FACE}'
```

```
This is a winking face: \\N{WINKING FACE}
```

In the end, you need to exercise care when processing raw text because raw text really isn't all that easy to process. It contains all sorts of extra bits of information that your application may not know how to handle.

Storing processed text data in sparse matrices

Feature extraction, as described in the “[Performing feature extraction](#)” section of the chapter, can produce relatively large matrices full of mostly zeros that would take considerable space to store. Compressing these matrices for storage would make sense because otherwise you'll spend considerable resources using them. Of course, you have to eventually expand the sparse matrices as well, so you need to know how to go both directions. The following sections discuss the use of sparse matrices.

Creating a sparse matrix

Fortunately, Python makes creating a sparse matrix easy. All you need is the `scipy.sparse` package, described at <https://docs.scipy.org/doc/scipy/reference/sparse.html>, which offers a number of sparse matrix conversions. The conversion you choose depends on how you want to see the sparse matrix afterward. This example relies on a row-based sparse matrix using the following code:

```
from scipy.sparse import csr_matrix
```

```
full_matrix = vectorized_text.todense()
```

```
print(full_matrix)
```

```
sparse_matrix = csr_matrix(full_matrix)
```

```
print(sparse_matrix)
```

In this case, you see both the expansion and the compression of the data. The `vectorized_text` variable actually contains a sparse matrix to begin with, but using `todense()` forms it into the matrix shown here, which is the decompressed (non-sparse) version:

```
[[0 0 0 1 0 0 0 0 1 0 1 0 0 0 1 0 1 0 1 0 0 0 2 0]]
```

```
[0 1 0 0 0 0 1 0 1 1 0 1 0 1 0 0 0 1 1 0 1 0 0 0 0 0]
[1 0 0 0 0 0 0 1 1 0 0 1 1 0 0 0 1 0 0 0 0 1 1 1 1 1]
[0 0 1 1 1 1 0 0 2 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 0]]
```

To create a sparse matrix, you use one of the sparse matrix classes. This example uses the Compressed Sparse Row (CSR) matrix. The output looks like this (with the actual output appearing in a single column):

<i>Coordinate</i>	<i>Number of Entries</i>	<i>Coordinate</i>	<i>Number of Entries</i>
(0, 3)	1	(0, 8)	1
(0, 10)	1	(0, 14)	1
(0, 16)	1	(0, 18)	1
(0, 20)	1	(0, 24)	2
(1, 1)	1	(1, 6)	1
(1, 8)	1	(1, 9)	1
(1, 11)	1	(1, 13)	1
(1, 17)	1	(1, 18)	1
(1, 20)	1	(2, 0)	1
(2, 7)	1	(2, 8)	1
(2, 11)	1	(2, 12)	1
(2, 16)	1	(2, 21)	1
(2, 22)	1	(2, 23)	1

<i>Coordinate</i>	<i>Number of Entries</i>	<i>Coordinate</i>	<i>Number of Entries</i>
(2, 24)	1	(2, 25)	1
(3, 2)	1	(3, 3)	1
(3, 4)	1	(3, 5)	1
(3, 8)	2	(3, 11)	1
(3, 15)	1	(3, 17)	1
(3, 19)	1		

As shown in the table, each entry in the full matrix becomes a set of coordinates and the value at the coordinate. Remember that the coordinates are zero-based, so entry (0, 3) is actually the fourth entry in the first row of the original matrix, which had a value of 1, as shown in the table.

Using the MovieLens sparse matrix

The MovieLens site (<https://movielens.org/>) is all about helping you find a movie you might like. After all, with millions of movies out there, finding something new and interesting could take time that you don't want to spend. The setup works by asking you to input ratings for movies that you already know about. The MovieLens site then makes recommendations for you based on your ratings. In short, your ratings teach an algorithm what to look for, and then the site applies this algorithm to the entire dataset.

You can obtain the MovieLens dataset directly at <https://grouplens.org/datasets/movielens/>, but the following code obtains the file for you automatically. (Note that the `url` is on multiple lines using a line continuation character, `\` or backslash, and that the second line must appear outdented as shown, or you can place the entire URL on a single line.)

```
import urllib.request
```

```
import os.path

from zipfile import ZipFile

filename = "ml-20m.zip"

if not os.path.exists("ml-20m.zip"):

    url = "https://files.grouplens.org/datasets/\
    movielens/ml-20m.zip"

    urllib.request.urlretrieve(url, filename)

archive = ZipFile(filename)

archive.extractall()
```

The first part of the code retrieves the MovieLens datasets from its online location using `urllib.request`. You must assign it a local filename, which is simply the same name used online in this case. Because MovieLens actually contains a number of datasets, the files appear as `.csv` (Comma Separated Value) files in a `.zip` archive. You must extract these files using the `extractall()` method of the `ZipFile` package, which creates a subdirectory in your code directory named `ml-20m`. This subfolder contains the six datasets and a `README.txt` file.

The interesting thing about this site is that you can download all or part of the dataset based on how you want to interact with it. You can find downloads in the following sizes:

- 100,000 ratings from 1,000 users on 1,700 movies
- 1 million ratings from 6,000 users on 4,000 movies
- 10 million ratings and 100,000 tag applications applied to 10,000 movies by 72,000 users
- 20 million ratings and 465,000 tag applications applied to 27,000 movies by 138,000 users

- MovieLens's latest dataset in small or full sizes (At this writing, the full size contained 21,000,000 ratings and 470,000 tag applications applied to 27,000 movies by 230,000 users; its size will increase in time.)

This dataset presents you with an opportunity to work with user-generated data using both supervised and unsupervised techniques. The large datasets present special challenges that only big data can provide.

However, this example starts a little more simply by reading the data into the application and then seeing what it contains. The first step is to read in the ratings information, as shown here:

```
ratings = pd.read_csv("ml-20m/ratings.csv")
```

```
print(ratings.shape)
```

```
print(ratings.head())
```

Reading the data may take a few moments on your system. After it has been read, you can see the output shown here:

```
(20000263, 4)
```

	userId	movieId	rating	timestamp
--	--------	---------	--------	-----------

0	1	2	3.5	1112486027
---	---	---	-----	------------

1	1	29	3.5	1112484676
---	---	----	-----	------------

2	1	32	3.5	1112484819
---	---	----	-----	------------

3	1	47	3.5	1112484727
---	---	----	-----	------------

4	1	50	3.5	1112484580
---	---	----	-----	------------

The column names are important because you use them later to create larger datasets on which you can perform various kinds of analysis. This example uses only two of the datasets, the second of which is the movie names. The following code shows how to import this second dataset:

```
names = pd.read_csv("ml-20m/movies.csv")
```

```
print(names.shape)
```

```
print(names.head())
```

The output of this code appears like this:

```
(27278, 3)
```

movieId		title \
0	1	Toy Story (1995)
1	2	Jumanji (1995)
2	3	Grumpier Old Men (1995)
3	4	Waiting to Exhale (1995)
4	5	Father of the Bride Part II (1995)
		genres
0	Adventure Animation Children Comedy Fantasy	
1	Adventure Children Fantasy	
2	Comedy Romance	
3	Comedy Drama Romance	
4	Comedy	

Note that both datasets have a `movieId` column. You can use this column to create a larger dataset using the following code:

```
movie_data = pd.merge(names, ratings, on="movieId")
```

```
print(movie_data.shape)
```

```
print(movie_data.head())
```

The dataset `shape` will change, of course, but because `movieId` is repeated, the output has only one copy, producing six, rather than seven, columns, as shown here:

```
(20000263, 6)
```

	movieId	title	genres	\	
1	1	Toy Story (1995)	Adventure Animation Child...		
2	1	Toy Story (1995)	Adventure Animation Child...		
3	1	Toy Story (1995)	Adventure Animation Child...		
4	1	Toy Story (1995)	Adventure Animation Child...		
	userId	rating	timestamp		
0	3	4.0	944919407		
1	6	5.0	858275452		
2	8	4.0	833981871		
3	10	4.0	943497887		
4	11	4.5	1230858821		

At this point, you can perform various kinds of analysis. For example, you might want to know the average rating of each movie. The following code sorts the movies by name and then finds the mean of the ratings for each movie:

```
print(movie_data.groupby('title')['rating'].mean()
      .sort_values().head())
```

The code first groups the entries and then finds the mean of the entries by `title`. It then sorts the entries based on the `rating` and outputs the first five entries, as shown here:

title	
Magic Christmas Tree, The (1964)	0.5
Vampir (Cuadecuc, vampir) (1971)	0.5
Prisoner of Zenda, The (1979)	0.5
Late Great Planet Earth, The (1979)	0.5
Last Warrior, The (Last Patrol, The) (2000)	0.5
Name: rating, dtype: float64	

Sparse matrices enable you to provide a great deal of text in a raw text form for analysis. The `.csv` files used for this example qualify as a sort of raw text form in that they aren't processed in any manner other than to organize them. Often, organized raw text will require a great deal of manipulation before you can get anything useful out of it. Consider that MovieLens has four other `.csv` files that this example hasn't even touched and, even with these two datasets, you can combine the various pieces of information in many other ways.

Understanding Semantics Using Word Embeddings

Most of this chapter involves accessing textual data, forming it in the right way, determining what the words actually say, and performing statistical analysis while storing the information efficiently. All these techniques play into modern NLP approaches. However, you also need to consider the role that AI can play, most especially the neural networks used for deep learning.

Neural networks are incredibly fast at processing data and finding the right weights to achieve the best predictions, and so are all the deep learning layers you find discussed today, from Convolutional Neural Networks (CNNs) to Recurrent Neural Networks (RNNs). These neural networks have effectiveness limits based on the data they have to process,

such as normalizing data to allow a neural network to work properly or forcing its range of input values between 0 to +1 or -1 to +1 to reduce trouble when updating network weights.



REMEMBER Normalization is done internally to the network by using activation functions like $\tanh$, which squeezes values to appear in the range from -1 to $+1$

(<https://tex.stackexchange.com/questions/176101/plotting-the-graph-of-hyperbolic-tangent>), or by using specialized layers like BatchNormalization (<https://keras.io/layers/normalization/>), which apply a statistical transformation on values transferred from one layer to another.

Another kind of problematic data that a neural network finds difficult to handle is sparse data. You have sparse data when your data mostly consists of zero values, which is exactly what happens when you process textual data using frequency or binary encoding, even if you don't use TF-IDF. When working with sparse data, not only will the neural network have difficulties finding a good solution (as technically explained in these Quora answers at <https://www.quora.com/Why-are-deep-neural-net-works-so-bad-with-sparse-data>), but you'll also need to have a huge number of weights for the input layer because sparse matrices are usually quite wide (they have many columns).

Sparse data problems motivated the use of *word embeddings*, which is a way to transform a sparse matrix into a dense one. Word embeddings can reduce the number of columns in the matrix from hundreds of thousands to a few hundred. Also, they allow no zero values inside the matrix. The word embedding process isn't done randomly but is devised so that words get similar values when they have the same meaning or are found within the same topics. In other words, it's a complex mapping; each embedding column is a specialty map (or a scale, if you prefer) and the similar or related words gather near each other.



TIP Word embeddings aren't the only advanced technique that you can use to make deep learning solutions shine with unstructured text, but

they are the technique that recent research advancements are developing and enhancing the most. Recently, a series of pretrained networks appeared that make it even easier to model language problems because they take into account not just the word you want to embed but also the words around it. Using these pretrained networks allows a more precise modeling of words with multiple meanings depending on their context. For instance, one of the most promising is the Google Bidirectional Encoder Representations from Transformers (BERT). Here's a link to the Google AI blog post describing the technique:

<https://ai.googleblog.com/2018/11/open-sourcing-bert-state-of-art-pre.html>.

As another example, you can have an embedding that transforms the name of different foods into columns of numeric values, which is a matrix of embedded words. On that matrix, the words that show fruits can have a similar score on a particular column. On the same column, vegetables can get different values, but not too far from those of fruit. Finally, the names of meat dishes can be far away in value from fruits and vegetables. An embedding performs this work by converting words into values in a matrix. The values are similar when the words are synonymous or refer to a similar concept. (This is called *semantic similarity*, with *semantic* referring to the meaning of words.)



REMEMBER Because the same semantic meaning can occur across languages, you can use carefully built embeddings to help you translate from one language to another: A word in one language will have the same embedded scores as the same word in another language. Researchers at Facebook AI Research (FAIR) lab have found a way to synchronize different embeddings and leverage them to provide multilingual applications based on deep learning (go to <https://code.fb.com/ml-applications/under-the-hood-multilingual-embeddings/> for details).

An important aspect to keep in mind when working with word embeddings is that they are a product of data and thus reflect the content of the data used to create them. Because word embeddings require large amounts of text examples for proper generation, the content of texts fed into the embeddings during the training is often retrieved automatically from the web and not fully scrutinized. The use of unverified input may

lead to word embeddings biases. For instance, you may be surprised to discover that the word embeddings create improper associations between words. You need to be aware of such a risk and test your application carefully because the consequence is adding the same unfair biases to the deep learning applications you create.

For now, the most popular word embeddings commonly used for deep learning applications are

- **Word2vec:** Created by a team of researchers led by Tomáš Mikolov at Google (you can read the original paper about this patented method here: <https://arxiv.org/pdf/1301.3781.pdf>). It relies on two shallow neural network layers that attempt to learn to predict a word by knowing the words that precede and follow it. Word2vec comes in two versions: one based on something like a bag-of-words model (called continuous bag-of-words, or CBoW), which is less sensitive to word order; and another based on n-grams (called continuous skip-gram), which is more sensitive to the order. Word2vec learns to predict a word given its context using *distributional hypothesis*, which means that similar words appear in similar contexts of words. By learning what words should appear in different contexts, Word2vec internalizes the contexts. Both versions are suitable for most applications, but the skip-gram version is actually better at representing infrequent words.
- **GloVe (Global Vectors):** Developed as an open source project at Stanford University (<https://nlp.stanford.edu/projects/glove/>), the GloVe approach is similar to statistical linguist methods. It takes word-word co-occurrence statistics from a corpus and reduces the resulting sparse matrix to a dense one using *matrix factorization*, which is an algebraic method widely used in multivariate statistics.
- **fastText:** Created by Facebook's AI Research (FAIR) lab, fastText (<https://fasttext.cc/>) is a word embedding, available in multiple languages that works with word subsequences instead of single words. It breaks a word down into many chunks of letters and embeds them. This technique has interesting implications because fastText offers a better representation of rare words (which are often composed of subsequences that aren't rare) and determines how to project misspelled words. The capability to handle misspellings and errors allows an effective use of the embedding with text coming from social networks, emails, and other sources people don't usually use a spell checker with.

EXPLAINING WHY (KING – MAN) + WOMAN = QUEEN

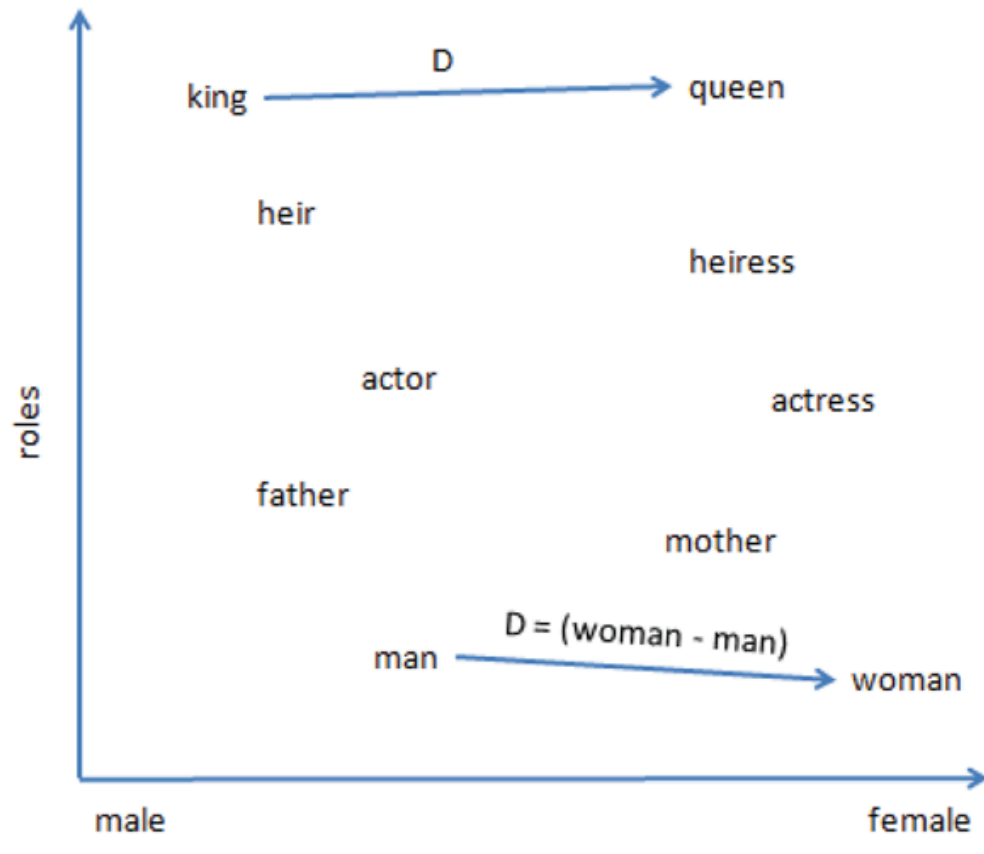
Word embeddings translate a word into a series of numbers representing its position in the embedding itself. This series of numbers is the *word vector*. It's usually made up of about 300 vectors (the number of vectors Google used in its model trained on the Google news dataset), and neural

networks use it to process textual information better and more effectively. In fact, words with similar meaning or that are used in similar contexts have similar word vectors; therefore, neural networks can easily spot words with similar meaning. In addition, neural networks can work with analogies by manipulating vectors, which means that you can obtain amazing results, such as

- king – man + woman = queen
- paris – france + poland = warsaw

It may seem like magic but it's simple mathematics. You can see how things work by looking at the following figure, which represents two Word2vec vectors.

Each vector in Word2vec represents a different semantic; it could be food type, quality of a person, nationality, or gender. There are many semantics and they aren't predefined; the embedding training created them automatically based on the presented examples. The figure shows two vectors from Word2vec: one representing the quality of a person, another representing the gender of a person. The first vector defines roles, starting with king and queen with higher scores, passing through actor and actress, and finally ending with man and woman having lower scores. If you add this vector to the gender vector, you see that the male and female variants separate by different scores on that vector. Now, when you subtract man and add woman to king, you are simply moving away from the coordinates of king and shifting along the gender vector until you reach the position of queen. This simple trick of coordinates, which doesn't imply any understanding of words by Word2vec, is possible because all the vectors of a word embedding are synchronized, representing the meaning of a language, and you can meaningfully shift from one coordinate to another as though you were shifting concepts in reasoning.



Using Scoring and Classification

The previous NLP discussions in this chapter show how a machine learning algorithm can read text (after scraping it from the web) using the BoW representation and how NLP can enhance its understanding of text using text length normalization, TF-IDF model, and n-grams. The following sections demonstrate how to put text processing into use by learning to solve two common problems in textual analysis: classification and sentiment analysis.

Performing classification tasks

When you classify texts, you assign a document to a class because of the topics it discusses. You can discover the topics in a document in different ways. The simplest approach is prompted by the idea that if a group of people talks or writes about a topic, the people tend to use words from a limited vocabulary because they refer or relate to the same topic. When you share some meaning or are part of the same group, you tend to use the same language. Consequently, if you have a collection of texts and don't know what topics the text references, you can reverse the previous reasoning; you can simply look for groups of words that tend to associate,

so their newly formed group by dimensionality reduction may hint at the topics you'd like to know about. This is a typical unsupervised learning task.

This learning task is a perfect application for the Singular Value Decomposition (SVD) family of algorithms, because by reducing the number of columns, the features (which, in a document, are the words) will gather in dimensions, and you can discover the topics by checking high-scoring words. SVD and Principal Components Analysis (PCA) provide features to relate both positively and negatively to the newly created dimensions. So a resulting topic may be expressed by the presence of a word (high positive value) or by the absence of it (high negative value), making interpretation both tricky and counterintuitive for humans. The `Scikit-learn` package includes the Non-Negative Matrix Factorization (NMF) decomposition class, which allows an original feature to relate only positively with the resulting dimensions.

This example starts by loading the `20newsgroups` dataset, a dataset collecting newsgroup postings scraped from the web, selecting only the posts regarding objects for sale and automatically removing headers, footers, and quotes. You may receive a warning message to the effect of `WARNING:sklearn.datasets.twenty_newsgroups:Downloading dataset from ...`, with the URL of the site used for the download when working with this code. The download will take a few moments because the dataset consumes 14MB.

```
import warnings

warnings.filterwarnings("ignore")

from sklearn.datasets import fetch_20newsgroups

dataset = fetch_20newsgroups(shuffle=True,

categories = ['misc.forsale'],

remove=('headers', 'footers', 'quotes'), random_state=101)

print ('Posts: %i' % len(dataset.data))
```

After the download is complete, you see the number of posts as output.

Posts: 585

The `TfidfVectorizer` class is imported and set up to remove stop words (common words such as *the* or *and*) and keep only distinctive words, producing a matrix whose columns point to distinctive words:

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
vectorizer = TfidfVectorizer(max_df=0.95,
```

```
min_df=2, stop_words='english')
```

```
tfidf = vectorizer.fit_transform(dataset.data)
```

```
from sklearn.decomposition import NMF
```

```
n_topics = 5
```

```
nmf = NMF(n_components=n_topics, random_state=101).fit(tfidf)
```



REMEMBER As noted earlier in the chapter, term frequency-inverse document frequency (TF-IDF) is a simple calculation based on the frequency of a word in a document. It is weighted by the rarity of the word among all the documents available. Weighting words is an effective way to rule out words that can't help you classify or identify the document when processing text. For example, you can eliminate common parts of speech or other common words.

As with other algorithms from the `sklearn.decomposition` module, the `n_components` parameter indicates the number of desired components. If you want to look for more topics, you use a higher number. As the required number of topics increases, the `reconstruction_err_` method reports lower error rates. It's up to you to decide when to stop, given the trade-off between more time spent on computations and more topics.

The last part of the script outputs the resulting five topics, as shown here:

```
feature_names = vectorizer.get_feature_names()
```

```
n_top_words = 15

for topic_idx, topic in enumerate(nmf.components_):

    print ("Topic #{}:" % (topic_idx+1),)

    print (" ".join([feature_names[i] for i in

                    topic.argsort()[: -n_top_words - 1: -1]]))
```

By reading the printed words, you can decide on the meaning of the extracted topics, thanks to product characteristics (for instance, the words *drive*, *hard*, *card*, and *floppy* refer to computers) or the exact product (for instance, *comics*, *car*, *stereo*, *games*):

Topic #1:

drive hard card floppy monitor meg ram disk motherboard

vga scsi brand color internal modem

Topic #2:

00 50 dos 20 10 15 cover 1st new 25 price man 40 shipping

comics

Topic #3:

condition excellent offer asking best car old sale good

new miles 10 000 tape cd

Topic #4:

email looking games game mail interested send like thanks

price package list sale want know

Topic #5:

shipping vcr stereo works obo included amp plus great

```
volume vhs unc mathes gibbs radley
```

You can explore the resulting model by looking into the attribute `components_` from the trained `NMF` model. It consists of a NumPy `ndarray` holding positive values for words connected to the topic. By using the `argsort` method, you can get the indexes of the top associations, whose high values indicate that they are the most representative words.

```
print (nmf.components_[0,:].argsort()[::-n_top_words-1:-1])
```

Here are the indexes of the top words for topic 0:

```
[1337 1749  889 1572 2342 2263 2803 1290 2353 3615 3017  806
1022 1938
```

```
2334]
```

Decoding the words' indexes creates readable strings by calling them from the array derived from the `get_feature_names` method applied to the `TfidfVectorizer` that was previously fitted.

```
print (vectorizer.get_feature_names()[1337])
```

For example, here is the human-readable form of index 1337:

```
drive
```

Analyzing reviews from e-commerce

Sentiment is difficult to catch because humans use the same words to express even opposite sentiments. The expression you convey is a matter of how you construct your thoughts in a phrase, not simply the words used. Even though dictionaries of positive and negative words do exist and are helpful, they aren't decisive because word context matters. You can use these dictionaries as a way to enrich textual features, but you have to rely more on machine learning if you want to achieve good results.



TIP

Seeing how positive and negative word dictionaries work is a good idea. The AFINN-111 dictionary contains 2,477 positive and negative

words and phrases

(http://www2.imm.dtu.dk/pubdb/views/publication_details.php?id=6010). Another good choice is the larger opinion lexicon by Hu and Liu that appears at <https://www.cs.uic.edu/~liub/FBS/sentiment-analysis.html#lexicon>. Both dictionaries contain English words.

Using labeled examples that associate phrases to sentiments can create more effective predictors. In this example, you create a machine learning model based on a dataset containing reviews from Amazon, Yelp, and IMDb that you can find at the UCI, the machine learning repository, <https://archive.ics.uci.edu/ml/datasets/Sentiment+Labelled+Sentences>.

This dataset was created for the paper “From Group to Individual Labels Using Deep Features,” by Kotzias and others, for KDD 2015. The dataset contains 3,000 labeled reviews equally divided from the three sources, and the data has a simple structure. Some text is separated by a tab from a binary sentiment label, where 1 is a positive sentiment and 0 a negative one. You can download the dataset and place it in your Python working directory using the following commands:

```
import urllib.request as urllib2
```

```
import requests, io, os, zipfile
```

```
UCI_url = 'https://archive.ics.uci.edu/ml/'
```

```
machine-learning-databases/00331/sentiment%20\
```

```
labelled%20sentences.zip'
```

```
response = requests.get(UCI_url)
```

```
compressed_file = io.BytesIO(response.content)
```

```
z = zipfile.ZipFile(compressed_file)
```

```
print ('Extracting in %s' % os.getcwd())
```

```

for name in z.namelist():

    filename = name.split('/')[ -1]

    nameOK = ('MACOSX' not in name and '.DS' not in name)

    if filename and nameOK:

        newfile = os.path.join(os.getcwd(),

                                os.path.basename(filename))

        with open(newfile, 'wb') as f:

            f.write(z.read(name))

        print ('\tunzipping %s' % newfile)

```



TIP In case the previous script doesn't work, you can download the data (in ZIP format) directly from <https://archive.ics.uci.edu/ml/machine-learning-databases/00331/> and expand it using your favorite unzipper. You'll find the `imdb_labelled.txt` file inside the newly created `sentiment labelled sentences` directory. After downloading the files, you can upload the IMDb file to a `pandas DataFrame` by using the `read_csv` function:

```

import numpy as np

import pandas as pd

dataset = 'imdb_labelled.txt'

data = pd.read_csv(dataset, header=None, sep=r"\t",

                    engine='python')

data.columns = ['review', 'sentiment']

```

Exploring the textual data is quite interesting. You'll find all short phrases such as "Wasted two hours" or "It was so cool." Some are clearly ambigu-

ous for a computer, such as “Waste your money on this game.” Even though waste has a negative meaning, the imperative makes the phrase sound positive. A machine learning algorithm can learn to decipher ambiguous phrases like these only after seeing many variants. The next step is to build the model by splitting the data into training and test sets:

```
from sklearn.cross_validation import train_test_split

corpus, test_corpus, y, yt = train_test_split(

    data.ix[:,0], data.ix[:,1],

    test_size=0.25, random_state=101)
```

After splitting the data, the code transforms the text using most of the NLP techniques described in this chapter: token counts, unigrams and bigrams, stop words removal, text length normalization, and TF-IDF transformation.

```
from sklearn.feature_extraction import text

vectorizer = text.CountVectorizer(ngram_range=(1,2),

    stop_words='english').fit(corpus)

TfidfF = text.TfidfTransformer()

X = TfidfF.fit_transform(vectorizer.transform(corpus))

Xt = TfidfF.transform(vectorizer.transform(test_corpus))
```

After the text for both the training and test sets is ready, the algorithm can learn sentiment using a linear support vector machine. This kind of support vector machine supports L2 regularization, so the code must search for the best C parameter using the grid search approach:

```
from sklearn.svm import LinearSVC

from sklearn.grid_search import GridSearchCV

param_grid = {'C': [0.01, 0.1, 1.0, 10.0, 100.0]}
```

```
clf = GridSearchCV(LinearSVC(loss='hinge',
                             random_state=101), param_grid)

clf = clf.fit(X, y)

print ("Best parameters: %s" % clf.best_params_)
```

The output shows the best hyperparameter to use:

```
Best parameters: {'C': 1.0}
```

Now that the code has determined the best hyperparameter for the problem, you can test performance on the test set using the *accuracy measure*, the percentage of correct times that the code can guess the correct sentiment:

```
from sklearn.metrics import accuracy_score

solution = clf.predict(Xt)

print("Achieved accuracy: %0.3f" %
      accuracy_score(yt, solution))
```

The results indicate accuracy of higher than 80 percent, but determining which phrases tricked the algorithm into making a wrong prediction is interesting.

```
Achieved accuracy: 0.816
```

You can print the misclassified texts and consider what the learning algorithm is missing in terms of learning from text:

```
print(test_corpus[yt!=solution])
```

That's quite a bit of text (and you're not seeing all of it in the book):

```
601    There is simply no excuse for something this p...
```

```
32     This is the kind of money that is wasted prope...
```

887 At any rate this film stinks, its not funny, a...

668 Speaking of the music, it is unbearably predic...

408 It really created a unique feeling though.

413 The camera really likes her in this movie.

138 I saw "Mirrormask" last night and it was an un...

132 This was a poor remake of "My Best Friends Wed...

291 Rating: 1 out of 10.

904 I'm so sorry but I really can't recommend it t...

410 A world better than 95% of the garbage in the ...

55 But I recommend waiting for their future effor...

826 The film deserves strong kudos for taking this...

100 I don't think you will be disappointed.

352 It is shameful.

171 This movie now joins Revenge of the Boogeyman ...

814 You share General Loewenhielm's exquisite joy ...

218 It's this pandering to the audience that sabot...

168 Still, I do like this movie for it's empowerme...

479 Of course, the acting is blah.

31 Waste your money on this game.

805 The only place good for this film is in the ga...

127 My only problem is I thought the actor playing...

613 Go watch it!

764	This movie is also revealing.
107	I love Lane, but I've never seen her in a movi...
674	Tom Wilkinson broke my heart at the end... and...
30	There are massive levels, massive unlockable c...
667	It is not good.
823	I struggle to find anything bad to say about i...
739	What on earth is Irons doing in this film?
185	Highly unrecommended.
621	A mature, subtle script that suggests and occa...
462	Considering the relations off screen between T...
595	Easily, none other cartoon made me laugh in a ...
8	A bit predictable.
446	I like Armand Assante & my cable company's sum...
449	I won't say any more - I don't like spoilers, ...
715	Im big fan of RPG games too, but this movie, i...
241	This would not even be good as a made for TV f...
471	At no point in the proceedings does it look re...
481	And, FINALLY, after all that, we get to an end...
104	Too politically correct.
522	Rating: 0/10 (Grade: Z) Note: The Show Is So B...
174	This film has no redeeming features.
491	This movie creates its own universe, and is fa...

```
Name: review, dtype: object
```

Table of contents collapsed